

Desarrollo Web para portal de Inteligencia Analítica

Lección 4. Controles avanzados de Streamlit

Introducción

Streamlit es una herramienta de código abierto que convierte *scripts* de **Python** en aplicaciones web interactivas y visualmente atractivas impulsadas por datos. Es ideal para científicos de datos y desarrolladores que desean agilizar sus tareas de visualización de datos y aprendizaje automático.

Anteriormente, configuramos un ambiente virtual para crear aplicaciones en **Streamlit**, trabajamos con controles interactivos y conectamos datos en formato CSV. Pero todavía hay mucho por explorar.

En ese sentido, la visualización es crucial en el análisis de datos, pues permite mostrar información en formatos gráficos e interpretar resultados. Con **Streamlit**, puedes usar gráficos de barras, de líneas, circulares, y mapas interactivos para representar datos de forma clara y efectiva.

En esta lección aprenderemos a crear aplicaciones interactivas usando gráficos y mapas, además, utilizaremos autenticación de usuarios, manejo de variables de sesión y operaciones de *logout*.

Es momento de llevar tus aplicaciones al siguiente nivel, ¿comenzamos?



Integración y visualización de datos usando *forms*, gráficos y mapas

Streamlit admite diferentes bibliotecas de gráficos, como **Altair**, **Bokeh**, **Matplotlib**, incluso, **Plotly**. Además, soporta bibliotecas de gráficos interactivos como **Vega Lite** para gráficos en **2D**, y **Deck.gl** para mapas y gráficos en **3D**. Sin embargo, también proporciona algunos tipos de gráficos que son nativos de **Streamlit** como **st.line_chart**, para crear gráficos de líneas, o **st.area_chart** para gráficos de área.

Para comprender estas herramientas, utilizarás **Visual Studio Code**, así como la base de datos del **titanic**, misma que puedes descargar en el apartado de archivos adjuntos en la plataforma.

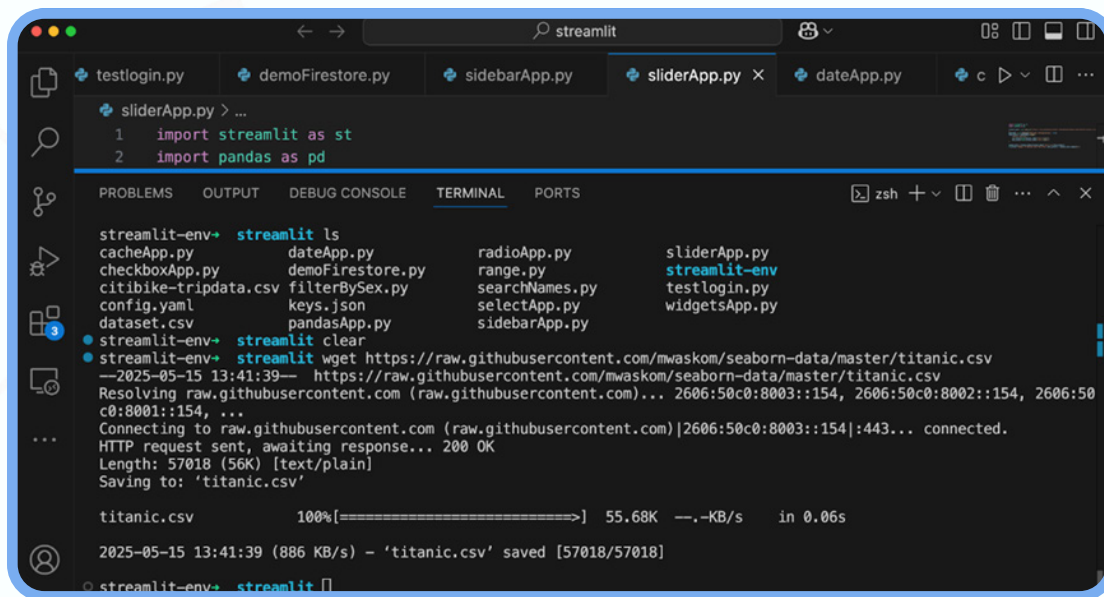
The image shows a screenshot of the Visual Studio Code editor interface. At the top, there are several tabs for different Python files: testlogin.py, demoFirestore.py, sidebarApp.py, sliderApp.py (which is the active tab), dateApp.py, and c. The sliderApp.py tab shows two lines of code: 'import streamlit as st' and 'import pandas as pd'. Below the code editor, there is a panel with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, and TERMINAL. The TERMINAL tab is selected, showing a terminal window with a zsh prompt. The terminal output shows the following commands and their results: 'streamlit ls' lists various files in the project; 'streamlit clear' clears the cache; 'streamlit wget https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv' downloads the Titanic dataset from GitHub. The download progress is shown as a bar chart, reaching 100% at 55.68 KB/s in 0.06s. The final output shows the file 'titanic.csv' saved successfully.

Figura 1. **Visual Studio Code**.

Cabe mencionar que, **Streamlit** también puede generar gráficos de la librería **Matplotlib**, una de las más populares y la primera librería de **Python** para la graficación de datos. Puedes encontrar más información sobre esta librería en el apartado de material adicional.

En esencia, cada aplicación web optimizada es una secuencia de comandos de **Python**. A continuación, revisarás cómo graficar con **Matplotlib** en **Streamlit**. Para hacerlo, sigue estos pasos:

Paso 1. En la parte superior del archivo, importa las dependencias necesarias, en este caso, utiliza las 3 principales librerías, como se muestra a continuación:

```
import pandas as pd

import streamlit as st

import matplotlib.pyplot as plt
```

Paso 2. Observa que utilizarás **Pandas** para la manipulación de los datos; **Streamlit** para construir la aplicación web, y **matplotlib.pyplot** para visualizar las gráficas. Recuerda que, usarás el **conjunto de datos del titanic**, que puedes descargar en el apartado de archivos adjuntos en la plataforma.

Paso 3. La primera sección que tendrá la aplicación web será la correspondiente al título y encabezado del sitio, para esto, utiliza:

```
# Create the title for the web app

st.title("My First Streamlit App")

st.header("Dataset Overview")
```

Paso 4. Despliega el contenido del conjunto de datos (*dataframe*) para que el usuario pueda visualizar la información que vas a graficar.

```
st.dataframe(titanic_data)
```

```
st.header("Data Description")
```

```
st.markdown("____")
```

Paso 5. Al hacerlo, observarás algo parecido a esto:

Dataset Overview									
	survived	pclass	sex	age	sibsp	parch	fare	embarked	c
0	0	3	male	22	1	0	7.2500	S	
1	1	1	female	38	1	0	71.2833	C	
2	1	3	female	26	0	0	7.9250	S	
3	1	1	female	35	1	0	53.1000	S	
4	0	3	male	35	0	0	8.0500	S	
5	0	3	male	NaN	0	0	8.4583	Q	
6	0	1	male	54	0	0	51.8625	S	
7	0	3	male	2	3	1	21.0750	S	
8	1	3	female	27	0	2	11.1333	S	
9	1	2	female	14	1	0	30.0708	C	S
10	1	3	female	4	1	1	16.7000	S	

Figura 2. Resumen de los datos.

Nota importante. La instrucción `st.markdown` ayuda a dividir las secciones con una línea y es útil para determinar cómo se dividirá la información dentro de la aplicación web.

Paso 6. Ahora que tienes la parte introductoria, **construye tu primera gráfica**: un histograma con las siguientes instrucciones:

```
fig, ax = plt.subplots()

ax.hist(titanic_data.fare)

st.header("Histograma del Titanic")

st.pyplot(fig)

st.markdown("____")
```

Observa que la primera instrucción inicializa la variable `fig` que contendrá el histograma, y la variable `ax` servirá para construirlo. La siguiente instrucción permite construir el histograma con el campo `fare` de la base de datos del **titanic**. La instrucción `st.header` sirve para poner un encabezado para la gráfica, mientras que, la instrucción `st.pyplot` ayuda a graficar el histograma en **Streamlit**. Finalmente, la instrucción `st.markdown` ayuda a generar una línea que sirva de división para la gráfica.

Paso 7. Ahora, genera una gráfica de barras. Para esto, utiliza dos campos del conjunto de datos, el campo `class` que tiene que ver con la clase de pasajeros del **titanic** y el campo `fare` que indica la tarifa que pago el pasajero. Para construir la gráfica solo necesitas las siguientes instrucciones:

```
fig2, ax2 = plt.subplots()
```

```

y_pos = titanic_data['class']

x_pos = titanic_data['fare']

ax2.barh(y_pos, x_pos)

ax2.set_ylabel("Class")

ax2.set_xlabel("Fare")

ax2.set_title('¿Cuanto pagaron las clases del
Titanic')

st.header("Grafica de Barras del Titanic")

st.pyplot(fig2)

st.markdown("___")

```

Observa que, en este caso, la primera instrucción inicializa la variable `fig` que contendrá la gráfica de barras, y la variable `ax` servirá para construirlo. Las variables `x_pos` y `y_pos` sirven para cargar los datos que corresponden a los campos `class` y `fare`, respectivamente, mismos que vas a pasarle a la gráfica de barras. Posteriormente, la instrucción `ax2.barh` permite generar una gráfica de barras horizontal que contendrá los campos antes mencionados. Las instrucciones `ax2.set_xlabel` y `ax2.set_ylabel` sirven para poner etiquetas para el eje de las **X** y el de las **Y**. Mientras que, la instrucción `ax2.set_title` ayuda a poner un título para la gráfica, y la instrucción `st.header`, como ya has estudiado, sirve para colocar un título para la gráfica. La instrucción `st.pyplot` le indica a **Streamlit** que gráfica visualizará. Finalmente, la instrucción `st.markdown` ayuda a generar una línea que sirve de división para la gráfica.

Paso 8. La última gráfica que vas a realizar es de dispersión, para esto, utiliza los campos `age` que corresponde a la edad de los pasajeros, y `fare` a la tarifa que pagaron los pasajeros del **titanic**. Utiliza el siguiente código para diseñar dicha gráfica:

```
fig3, ax3 = plt.subplots()

ax3.scatter(titanic_data.age, titanic_data.fare)

ax3.set_xlabel("Edad")

ax3.set_ylabel("Tarifa")

st.header("Grafica de Dispersión del Titanic")

st.pyplot(fig3)
```

La primera instrucción inicializa la variable `fig` que contendrá la gráfica de dispersión, y la variable `ax` sirve para construirlo. La instrucción `ax3.scatter` de Matplotlib permite construir una gráfica de dispersión, y recibe dos parámetros, en este caso, los campos "age" y "fare". Las instrucciones `ax3.set_xlabel` y `ax3.set_ylabel` permiten colocar etiquetas en el eje de las **X** y en el eje de las **Y**. Ahora, vamos a utilizar la instrucción `st.header` para colocar un título para la gráfica. Finalmente, la instrucción `st.pyplot` le indica a **Streamlit** que visualice la gráfica que deseas.

Hasta el momento has revisado el funcionamiento de tu aplicación web y cómo puedes diseñar diferentes tipos de gráficos. Para un mayor entendimiento, continúa con los pasos:

Paso 9. En **Visual Studio Code**, crea **demoGraph.py** y edítalo con el siguiente código:

```
$ touch demoGraph.py

import pandas as pd

import matplotlib.pyplot as plt

import streamlit as st

titanic_link = 'titanic.csv'

titanic_data = pd.read_csv(titanic_link)

fig, ax = plt.subplots()

ax.hist(titanic_data.fare)

st.header("Histograma del Titanic")

st.pyplot(fig)

st.markdown("___")

fig2, ax2 = plt.subplots()

y_pos = titanic_data['class']
```

```
x_pos = titanic_data['fare']

ax2.barh(y_pos, x_pos)

ax2.set_ylabel("Class")

ax2.set_xlabel("Fare")

ax2.set_title('¿Cuánto pagaron las clases del
Titanic')

st.header("Gráfica de Barras del Titanic")

st.pyplot(fig2)

st.markdown("____")

fig3, ax3 = plt.subplots()

ax3.scatter(titanic_data.age, titanic_data.fare)

ax3.set_xlabel("Edad")

ax3.set_ylabel("Tarifa")

st.header("Gráfica de Dispersión del Titanic")

st.pyplot(fig3)
```

Paso 10. Activa el ambiente virtual que instalaste en **Matplotlib**:

```
$ source streamlit-env/bin/activate
```

```
$ pip install matplotlib
```

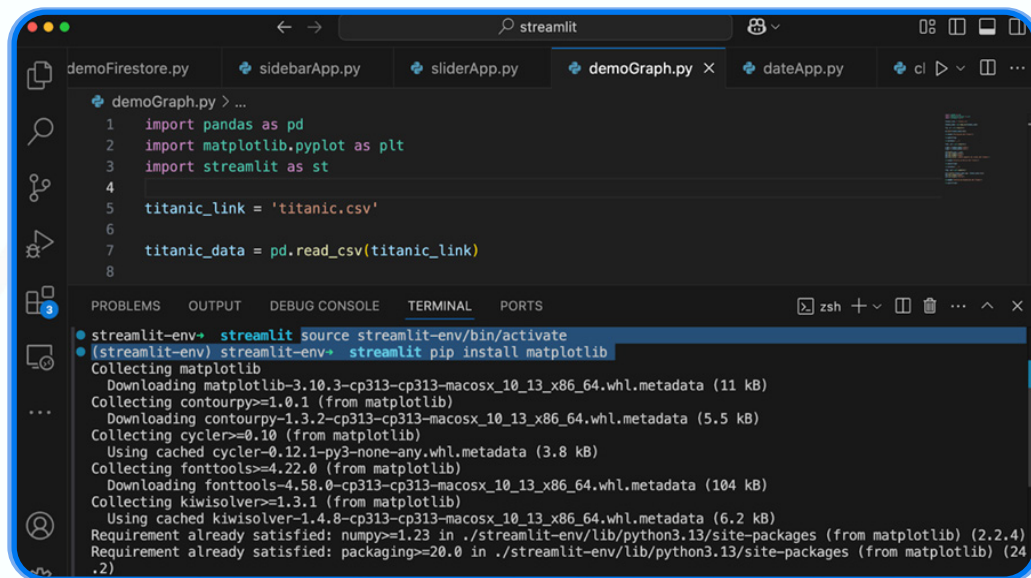


Figura 3. Ambiente virtual.

Paso 11. Ejecuta la aplicación, y al hacerlo, se cargará una pestaña en tu navegador predeterminado. Visualiza la aplicación web que has creado.

```
$ streamlit run demoGraph.py
```

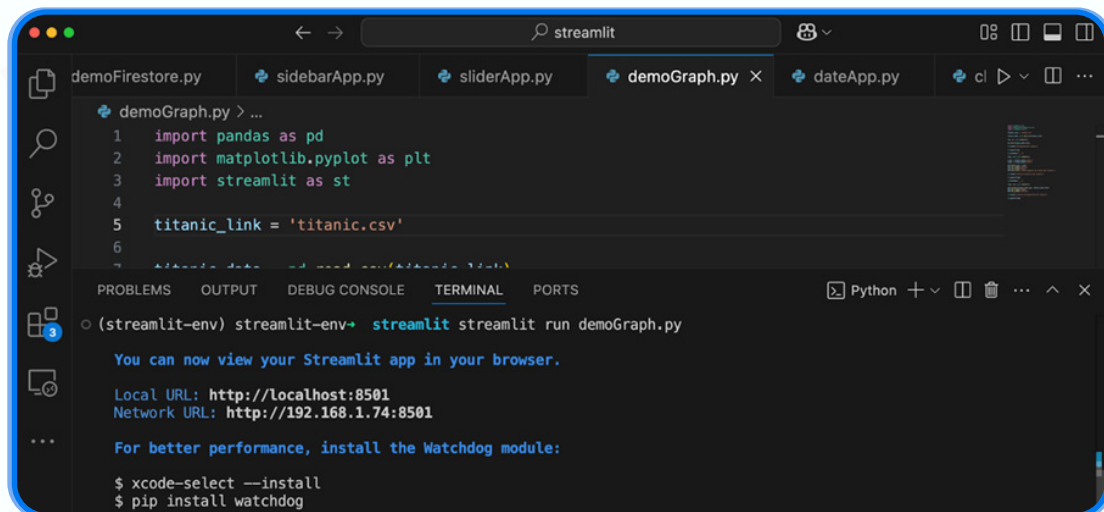


Figura 4. Edición de **demoGraph.py**.

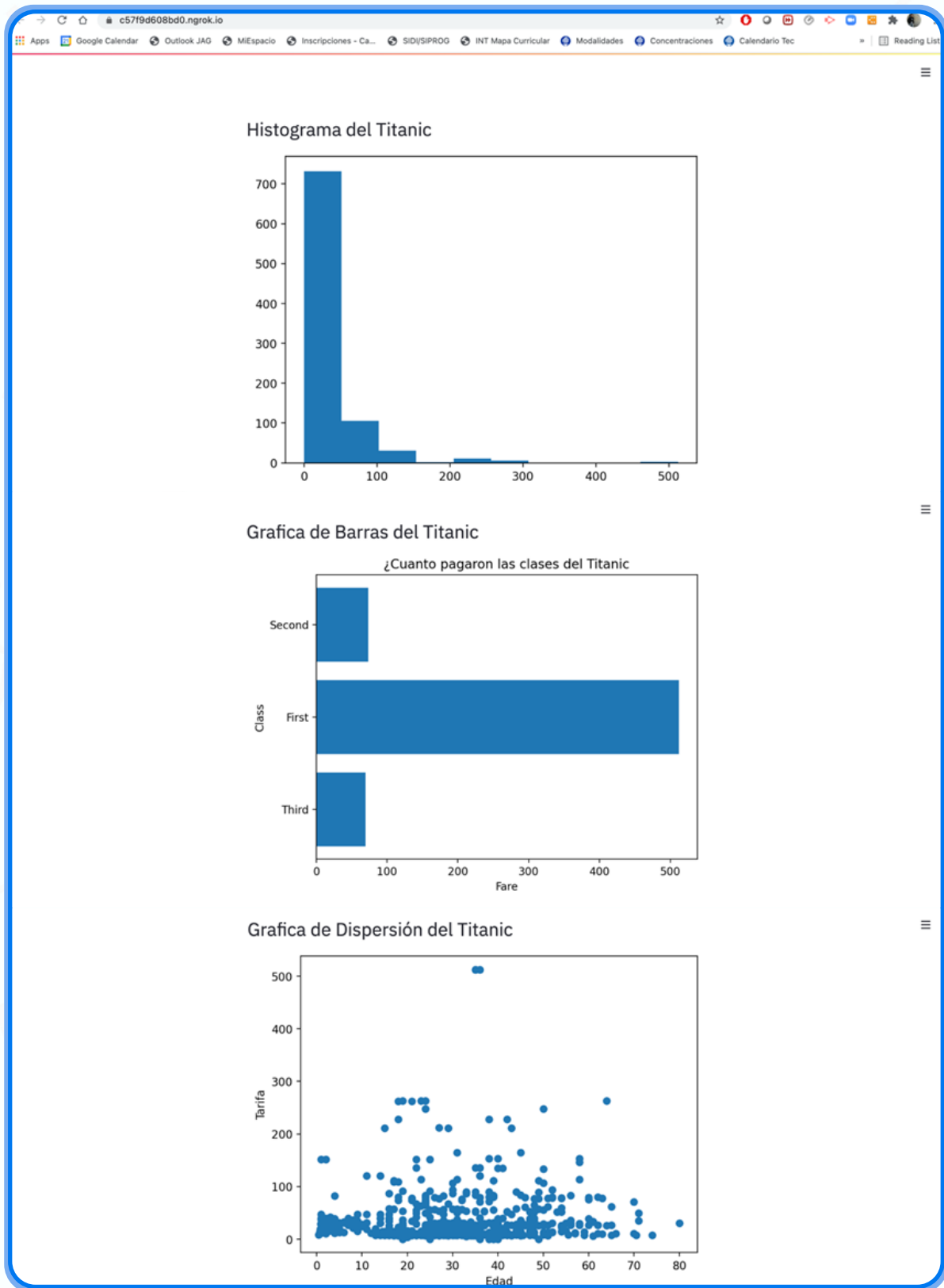


Figura 5. Vista de la aplicación web con **Streamlit** y **Google Colab** con la visualización de las gráficas.

Hasta ahora has aprendido a graficar en **Streamlit** para una aplicación web, estos procedimientos son sumamente útiles en tus funciones como científico de datos. De manera complementaria, te invitamos a que revises el siguiente enlace donde encontrarás las instrucciones que se requieren para contestar a las interrogantes planteadas a lo largo de estos contenidos: <https://docs.streamlit.io/en/stable/api.html#display-charts>.

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 06 de junio de 2025 y su disponibilidad depende del autor.

Streamlit y visualización de mapas

Streamlit usa un motor de mapas llamado **Mapbox**, una plataforma de código abierto para la creación de mapas utilizada principalmente por desarrolladores. Se basa en mapas vectoriales para diseñar y personalizar el estilo de los mapas según las necesidades de los usuarios. **Mapbox** está basado en un proyecto de *software* libre **OpenStreetMap**.

Ahora, con la función `st.map()` puedes desplegar puntos GPS sobre un mapa. Para hacerlo, sigue estos pasos:

Paso 1. En este primer ejemplo, utiliza **NumPy** para generar algunos datos como ejemplo, y pintarlos sobre un mapa de la ciudad de San Francisco.

```
map_data = pd.DataFrame(  
    np.random.randn(1000, 2) / [50, 50] + [37.76,  
    -122.4],  
    columns=['lat', 'lon'])  
st.map(map_data)
```

Paso 2. Observa que el *dataframe* creado tiene dos columnas llamadas `lat` y `lon`, las cuales son reconocidas por `st.map` como los puntos GPS que serán pintados en el mapa. Luego entonces, cualquier fuente de datos de una API, o bien de Excel que contenga las columnas `lat` y `lon` también serán reconocidas por `st.map()` como puntos GPS.

En esencia, cada aplicación web con **Streamlit** puede contener mapas sin costos de licenciamiento o pago por uso. En este ejemplo completo colocarás en la parte superior del archivo las dependencias necesarias. En este caso, utiliza las tres principales, como se muestra a continuación:

```
import pandas as pd

import numpy as np

import streamlit as st
```

Es decir, utilizarás **Pandas** para la manipulación de los datos; **Streamlit** para construir la aplicación web, y **NumPy** para la generación de los puntos GPS aleatorios.

```
map_data = pd.DataFrame(

    np.random.randn(1000, 2) / [50, 50] + [37.76,
-122.4],

    columns=['lat', 'lon'])
```

Paso 3. La primera sección de la aplicación web será la correspondiente al título y un encabezado del sitio web. Para hacerlo, utiliza:

```
# Create the title for the web app

st.title("San francisco Map")

st.header("Using Streamlit and Mapbox")
```

Paso 4. Despliega el conjunto de datos GPS para que el usuario pueda visualizar la información que vas a pintar en el mapa.

```
st.map(map_data)
```

Paso 5. En **Visual Studio Code**, crea **demoMap.py** y edítalo con el código completo:

```
import pandas as pd

import numpy as np

import streamlit as st

map_data = pd.DataFrame(

    np.random.randn(1000, 2) / [50, 50] + [37.76,

-122.4],

    columns=['lat', 'lon'])
```

```
# Create the title for the web app

st.title("San francisco Map")

st.header("Using Streamlit and Mapbox")

st.map(map_data)

$ touch demoMap.py
```

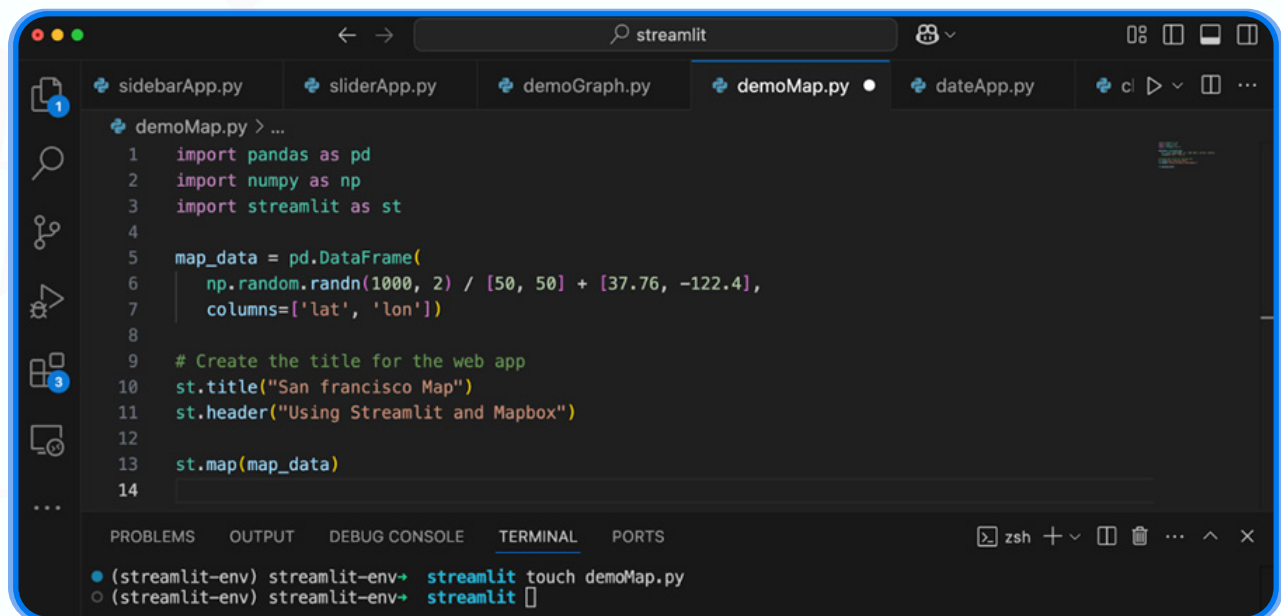


Figura 6. Edición de **demoMap.py**.

Paso 6. Al hacerlo, se cargará una pestaña en el navegador predeterminado. Visualiza la aplicación web que has creado.

```
$ streamlit run demoMap.py
```


En ese sentido, la estructura de la aplicación web debe:

- Incluir un título.
- Incluir un subtítulo (opcional).
- Incluir un breve párrafo que describa el objetivo de este proyecto de analítica de datos. Por ejemplo: Uso de bicicletas en la ciudad de Nueva York con filtros por hora.
- Incluir un control *slider* que permita filtrar los viajes por hora del día, usar la columna `Date/Time`.
- Incluir una sección que contenga el mapa.

Adicionalmente, **considera crear una función para la obtención de datos**. El ejemplo de la función es:

```
@st.cache

def load_data(nrows):

    data = pd.read_csv(DATA_URL, nrows=nrows)

    lowercase = lambda x: str(x).lower()

    data.rename({'start_lat': 'lat', 'start_lng':
'lon'}, axis=1, inplace=True)

    data[DATE_COLUMN] = pd.to_datetime(data[DATE_
COLUMN])

    return data
```


Debido a que la base de datos es muy grande, la sugerencia es recuperar solo una muestra: `data = load_data(10000)` # probar con 100, 1000, etcétera. Como la columna `Date/Time` es de tipo **Fecha y Hora**, la sugerencia es aplicar un filtro por hora:

```
filtered_data = data[data[DATE_COLUMN].dt.hour ==  
hour_to_filter]
```

Para continuar, sigue estos pasos:

Paso 7. En **Visual Studio Code** crea **demoNY.py**, y editalo con el siguiente código:

```
$ touch demoNY.py  
  
import streamlit as st  
  
import pandas as pd  
  
import numpy as np  
  
st.title('Cicle Rides in NYC')  
  
DATE_COLUMN = 'started_at'  
  
DATA_URL = ('citibike-tripdata.csv')  
  
@st.cache
```

```

def load_data(nrows):

    data = pd.read_csv(DATA_URL, nrows=nrows)

    lowercase = lambda x: str(x).lower()

    data.rename({'start_lat': 'lat', 'start_lng':
'lon'}, axis=1, inplace=True)

    data[DATE_COLUMN] = pd.to_datetime(data[DATE_
COLUMN])

    return data

data_load_state = st.text('Loading cicle nyc data...')

data = load_data(1000)

data_load_state.text("Done! (using st.cache)")

if st.sidebar.checkbox('Show raw data'):

    st.subheader('Raw data')

    st.write(data)

if st.sidebar.checkbox('Recorridos por hora'):

    st.subheader('Numero de recorridos por hora')

    hist_values = np.histogram(data[DATE_COLUMN].
dt.hour, bins=24, range=(0,24))[0]

    st.bar_chart(hist_values)

```

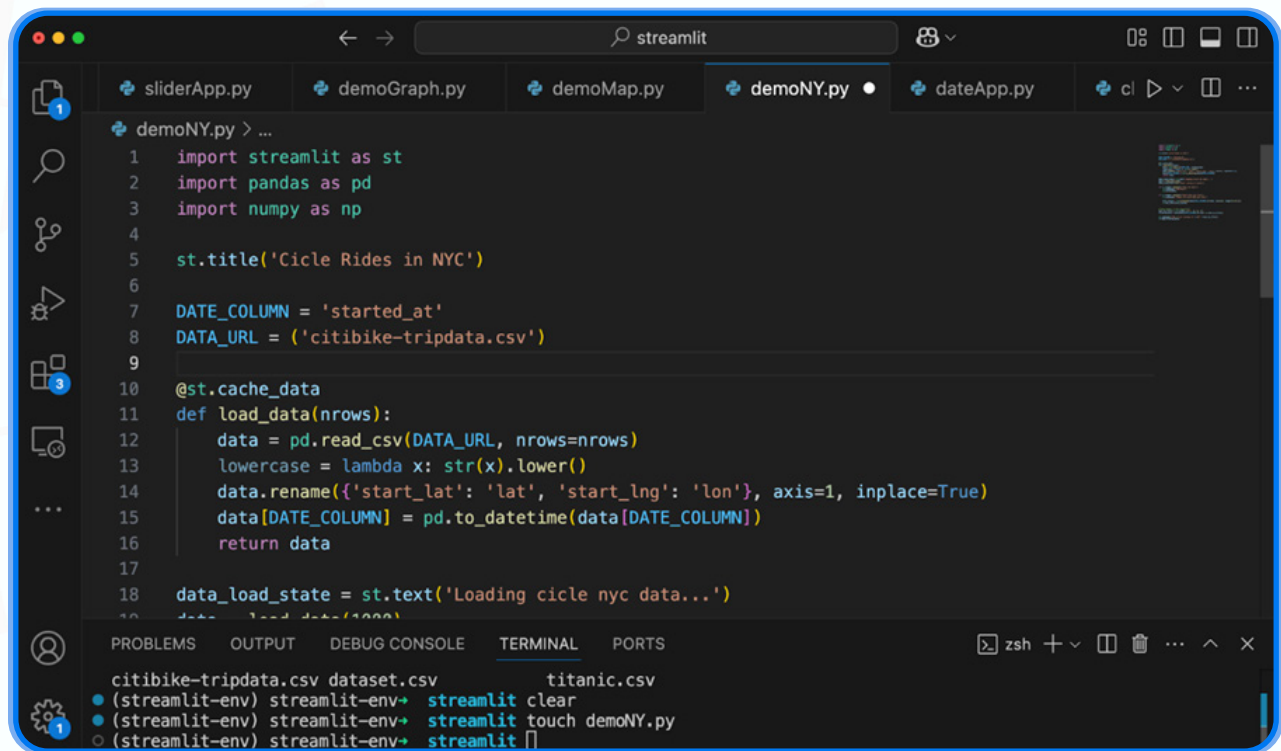
```
# Some number in the range 0-23
```

```
hour_to_filter = st.slider('hour', 0, 23, 17)
```

```
filtered_data = data[data[DATE_COLUMN].dt.hour ==  
hour_to_filter]
```

```
st.subheader('Map of all pickups at %s:00' % hour_to_  
filter)
```

```
st.map(filtered_data)
```

A screenshot of a Streamlit application editor interface. The top bar shows the file explorer with tabs for sliderApp.py, demoGraph.py, demoMap.py, demoNY.py (selected), and dateApp.py. The main editor area displays the code for demoNY.py, which includes imports for streamlit, pandas, and numpy, a title 'Cicle Rides in NYC', and a function to load and process CSV data. The bottom panel shows the terminal with commands to clear, touch, and run the application.

```
demoNY.py > ...  
1 import streamlit as st  
2 import pandas as pd  
3 import numpy as np  
4  
5 st.title('Cicle Rides in NYC')  
6  
7 DATE_COLUMN = 'started_at'  
8 DATA_URL = ('citibike-tripdata.csv')  
9  
10 @st.cache_data  
11 def load_data(nrows):  
12     data = pd.read_csv(DATA_URL, nrows=nrows)  
13     lowercase = lambda x: str(x).lower()  
14     data.rename({'start_lat': 'lat', 'start_lng': 'lon'}, axis=1, inplace=True)  
15     data[DATE_COLUMN] = pd.to_datetime(data[DATE_COLUMN])  
16     return data  
17  
18 data_load_state = st.text('Loading cicle nyc data...')  
19 data = load_data(10000)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

citibike-tripdata.csv dataset.csv titanic.csv
(streamlit-env) streamlit-env+ streamlit clear
(streamlit-env) streamlit-env+ streamlit touch demoNY.py
(streamlit-env) streamlit-env+ streamlit

Figura 9. Edición de **demoNY.py**.

Paso 8. Observa el ejemplo de salida: la vista con todos los registros visualizados.

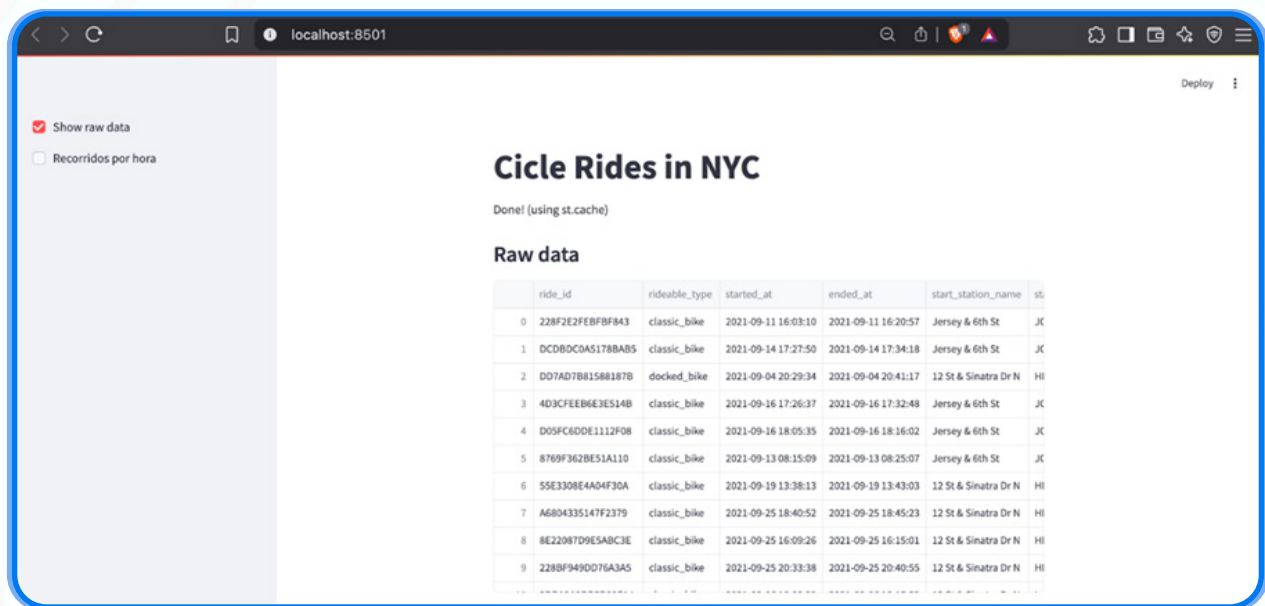


Figura 10. Vista con los registros visualizados.

También observa la vista con gráfico del número de viajes ocurridos por hora.



Figura 11. Vista con gráfico del número de viajes ocurridos por hora.

Además, tienes la vista con filtro para visualizar los recorridos por hora en un mapa.

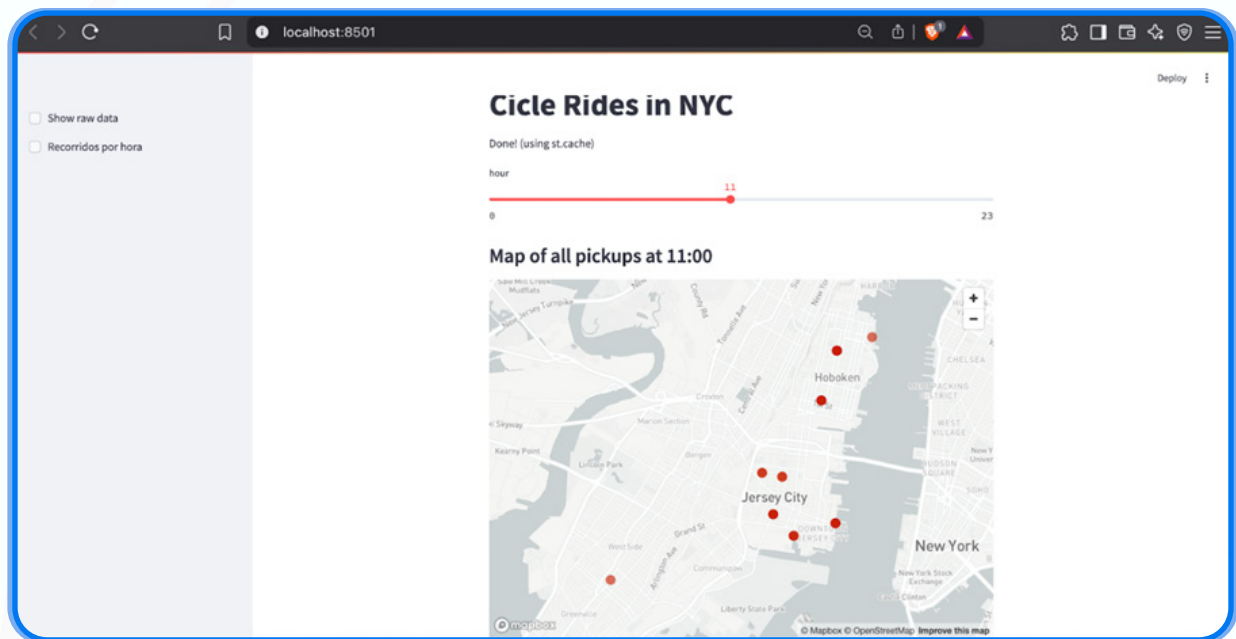


Figura 12. Vista con filtro para visualizar los recorridos por hora en un mapa.

En el campo de la ciencia de datos, la visualización juega un papel crucial para entender conjuntos de datos complejos y extraer ideas significativas. Tres herramientas poderosas que han revolucionado este campo son **Streamlit**, **Matplotlib** y **Map**. Las primeras son una potente combinación para crear visualizaciones de datos interactivas y paneles de control. La simplicidad y flexibilidad de **Streamlit** lo convierten en una excelente herramienta para construir aplicaciones web, mientras que la amplia gama de tipos de gráficos y características de interactividad de **Matplotlib** se traducen en una herramienta versátil para la visualización de datos.

Finalmente, la integración de **Streamlit** y **Plotly** abre un mundo de posibilidades para científicos de datos y desarrolladores. Desde la creación de simples gráficos de barras hasta la construcción de paneles de control complejos con múltiples visualizaciones interactivas, estas herramientas brindan las capacidades necesarias para convertir los datos en conocimientos.

Integración de operaciones de *login*, *logout*, manejo de caché y variables de sesión

El proceso de inicio de sesión (*login*) en un sitio web generalmente implica que un usuario ingrese su nombre de usuario y contraseña para autenticarse, y así, acceder a contenido restringido. Este proceso es esencial para sitios web que requieren registro y autenticación de los usuarios antes de navegar ciertas páginas o utilizar ciertas funciones. Una vez hecho este proceso, los usuarios pueden navegar por todas las páginas del sitio.

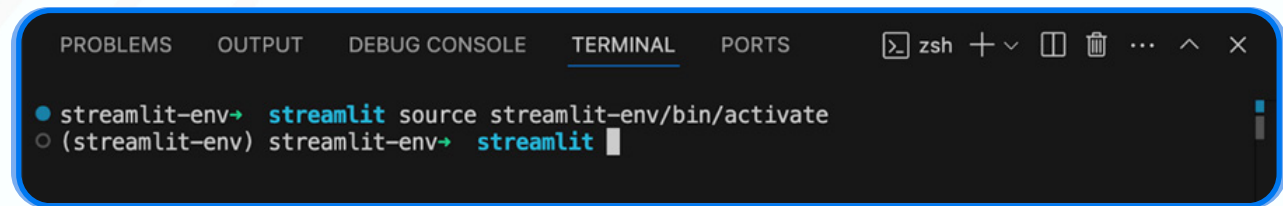
Login en Streamlit

Los sitios web pueden incluir diferentes métodos de inicio de sesión, además del tradicional ya mencionado. Por ejemplo, algunos sitios permiten a los usuarios iniciar sesión utilizando sus cuentas de redes sociales como Google, Facebook o Apple, simplificando el proceso y generando una sensación de inmediatez. Además, los sitios web pueden implementar medidas de seguridad adicionales, como la autenticación de dos factores, que requiere que el usuario proporcione dos formas de identificación antes de acceder al sitio.

En el desarrollo de cuadros de mando en **Streamlit**, regularmente se necesitan características como una autenticación básica de usuarios. En este momento, usarás un paquete de **Streamlit** llamado `streamlit-authenticator`, para autenticar usuarios.

En ese sentido, para construir tu aplicación con *login*, sigue estos pasos.

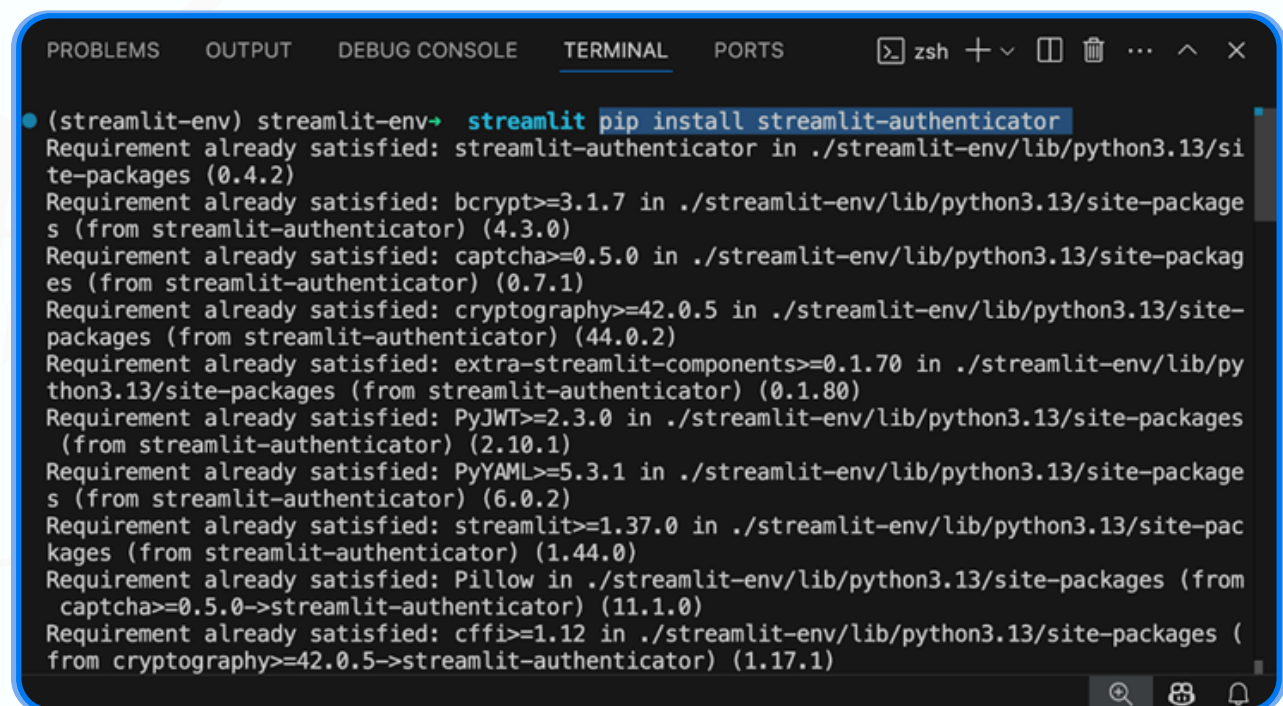
Paso 1. Instala Streamlit-Authenticator. Para esto, en **Visual Studio Code**, abre tu carpeta del proyecto y activa el ambiente virtual.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh + - [ ] [ ] ... ^ X
● streamlit-env→ streamlit source streamlit-env/bin/activate
○ (streamlit-env) streamlit-env→ streamlit
```

Figura 1. Proyecto en **Streamlit**.

Después, instala el paquete `streamlit-authenticator` con: `pip install streamlit-authenticator`



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh + - [ ] [ ] ... ^ X
● (streamlit-env) streamlit-env→ streamlit pip install streamlit-authenticator
Requirement already satisfied: streamlit-authenticator in ./streamlit-env/lib/python3.13/site-packages (0.4.2)
Requirement already satisfied: bcrypt>=3.1.7 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (4.3.0)
Requirement already satisfied: captcha>=0.5.0 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (0.7.1)
Requirement already satisfied: cryptography>=42.0.5 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (44.0.2)
Requirement already satisfied: extra-streamlit-components>=0.1.70 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (0.1.80)
Requirement already satisfied: PyJWT>=2.3.0 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (2.10.1)
Requirement already satisfied: PyYAML>=5.3.1 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (6.0.2)
Requirement already satisfied: streamlit>=1.37.0 in ./streamlit-env/lib/python3.13/site-packages (from streamlit-authenticator) (1.44.0)
Requirement already satisfied: Pillow in ./streamlit-env/lib/python3.13/site-packages (from captcha>=0.5.0->streamlit-authenticator) (11.1.0)
Requirement already satisfied: cffi>=1.12 in ./streamlit-env/lib/python3.13/site-packages (from cryptography>=42.0.5->streamlit-authenticator) (1.17.1)
```

Figura 2. Instalación.

Paso 2. Crea el archivo **config.yaml** con las credenciales que usarás en tu proyecto:

credentials:

 usernames:


```
admin:

  email: admin@gmail.com

  name: admin

  password: admin

test:

  email: test@gmail.com

  name: test

  password: test
```

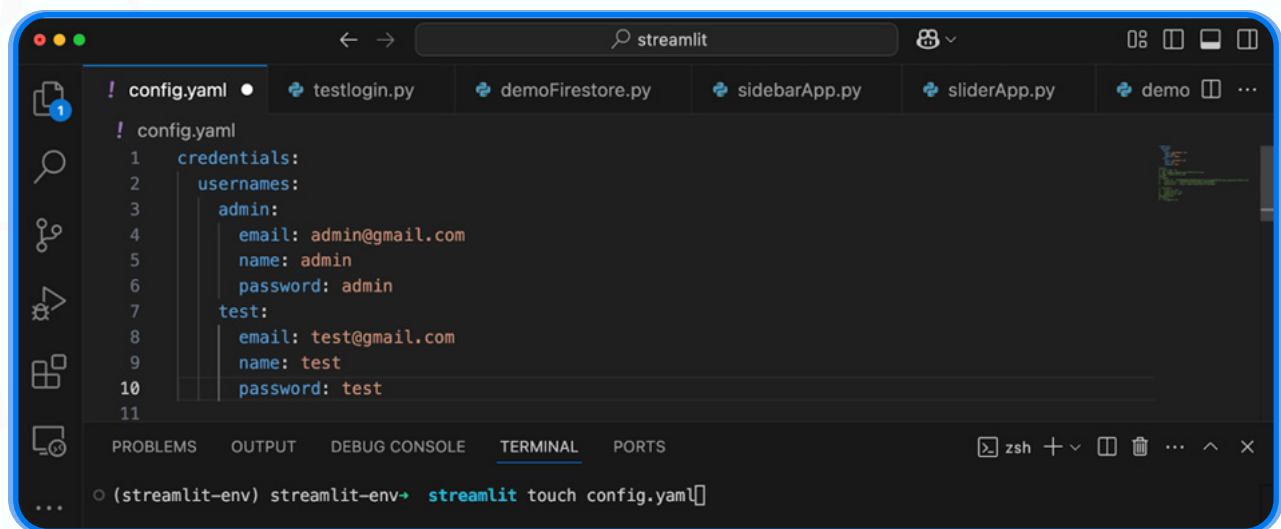


Figura 3. Archivo **config.yaml**.

Paso 3. Para el proyecto, utilizarás dos usuarios: **admin** y **test**. Cada uno con su respectiva contraseña: **admin** y **test**. En el siguiente paso, crearás el **testLogin.py** y editarás el código:

```
import streamlit as st

import streamlit_authenticator as stauth

import yaml

from yaml.loader import SafeLoader

from streamlit_authenticator.utilities import *

with open('config.yaml') as file:

    config = yaml.load(file, Loader=SafeLoader)

authenticator = stauth.Authenticate(

    config['credentials'],

    config['cookie']['name'],

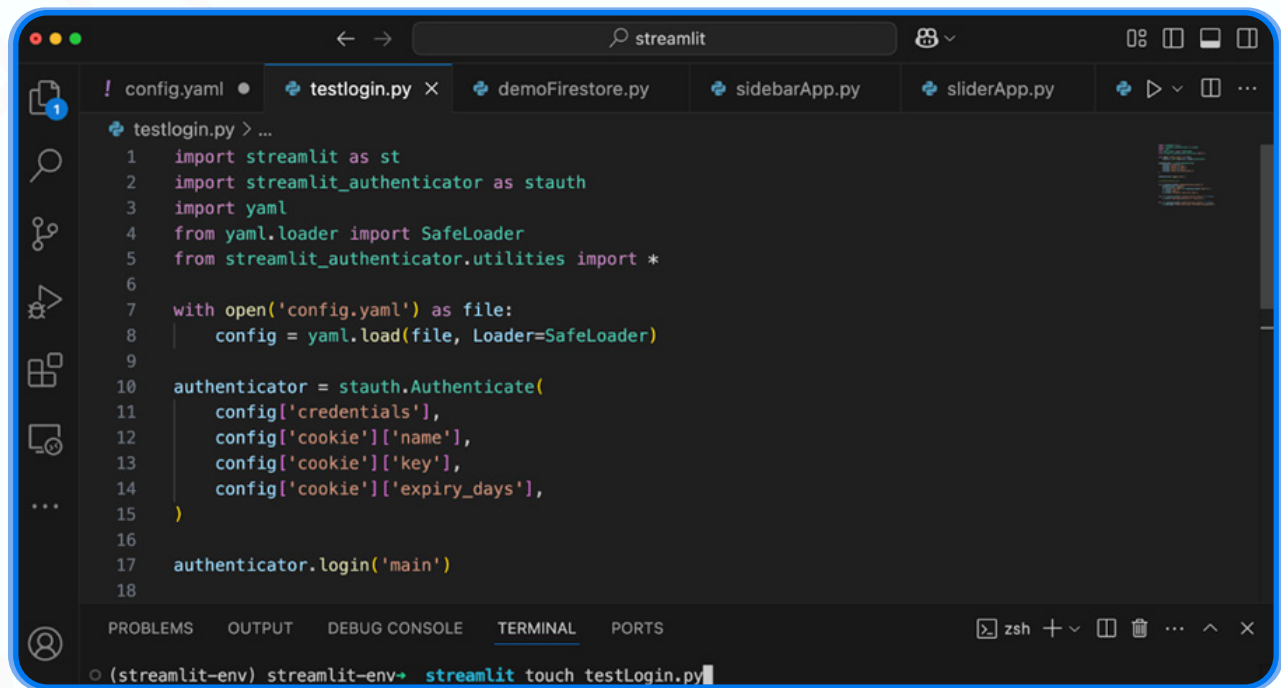
    config['cookie']['key'],

    config['cookie']['expiry_days'],

)

authenticator.login('main')
```

```
$ touch testLogin.py
```



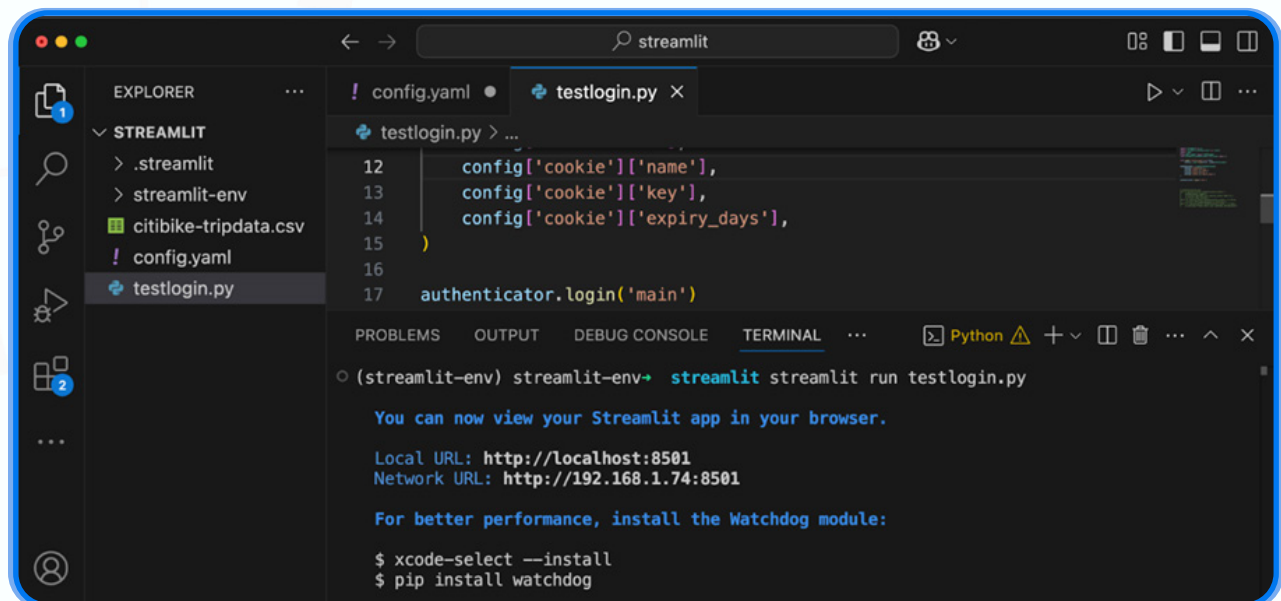
The screenshot shows the VS Code editor interface with the Streamlit environment. The file explorer on the left shows the project structure. The main editor displays the testlogin.py file with the following code:

```
1 import streamlit as st
2 import streamlit_authenticator as stauth
3 import yaml
4 from yaml.loader import SafeLoader
5 from streamlit_authenticator.utilities import *
6
7 with open('config.yaml') as file:
8     config = yaml.load(file, Loader=SafeLoader)
9
10 authenticator = stauth.Authenticate(
11     config['credentials'],
12     config['cookie']['name'],
13     config['cookie']['key'],
14     config['cookie']['expiry_days'],
15 )
16
17 authenticator.login('main')
18
```

The terminal at the bottom shows the command: `(streamlit-env) streamlit-env➤ streamlit touch testlogin.py`

Figura 4. Edición de código.

Paso 4. Prueba tu aplicación con: `streamlit run testlogin.py`



The screenshot shows the VS Code editor interface with the Streamlit environment. The file explorer on the left shows the project structure. The main editor displays the testlogin.py file with the following code:

```
12 config['cookie']['name'],
13 config['cookie']['key'],
14 config['cookie']['expiry_days'],
15 )
16
17 authenticator.login('main')

```

The terminal at the bottom shows the command: `(streamlit-env) streamlit-env➤ streamlit run testlogin.py`

The output in the terminal is:

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.74:8501

For better performance, install the Watchdog module:

$ xcode-select --install
$ pip install watchdog
```

Figura 5. Ejecución de la aplicación.

Paso 5. Al hacerlo, visualizarás tu aplicación en **localhost:8501**, con el formulario de **login** listo para ingresar las credenciales de **admin** o **test**.

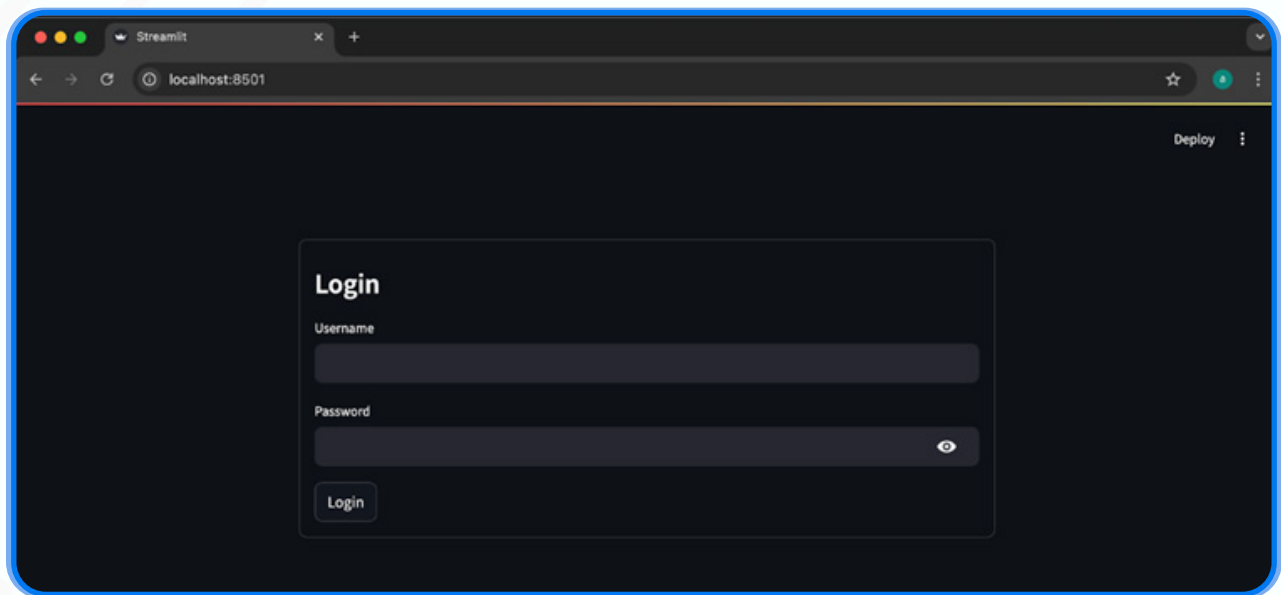


Figura 6. Vista de la aplicación.

Hasta ahora, has creado una aplicación simple que permite capturar las credenciales de acceso con usuario y contraseña. Continúa para profundizar en las variables de sesión que sirven para administrar el usuario y estado de la aplicación.

Variables de sesión en Streamlit

Las variables de sesión en **Streamlit** son un método para almacenar variables disponibles en múltiples páginas sin tener que pasarlas como parámetro. Las variables de sesión se almacenan en el servidor y tienen un tiempo limitado de existencia. Para identificar al usuario que generó las variables de sesión, el servidor genera una clave única que es enviada al navegador y almacenada en una *cookie* (clave única). Después, cada vez que el navegador solicita otra página al mismo sitio, envía esta *cookie* con la cual el servidor identifica de qué navegador proviene la petición y puede rescatar las variables de sesión que se han creado.

En esencia, las variables de sesión caducan si durante algunos minutos, regularmente, un intervalo de 10 a 20 minutos, no se detecta ninguna actividad en el sitio por parte de ese visitante específico del sitio, o si este abandona su navegador web. Sin embargo, es posible configurar la propiedad `timeout` del objeto `session` para que las variables de sesión existan por un tiempo diferente.

Cabe mencionar que las variables de sesión son más seguras que las *cookies*, ya que se almacenan en el servidor y no tienen que estar enviándose continuamente como sucede con estas.

Continuando con la explicación de estos conceptos, agregarás funcionalidad a tu aplicación **testLogin.py**, para así, implementar variables de sesión que ayuden a administrar el estado de la aplicación y validar el usuario activo. Para hacerlo, sigue estos pasos:

Paso 1. En **Visual Studio Code**, agrega el código al programa **testLogin.py**.

```
# Authenticating user

if st.session_state['authentication_status']:

    authenticator.logout()

    st.write(f'Welcome *{st.session_state["name"]}*')

    st.title('Home')

    st.write('Streamlit app with login')

elif st.session_state['authentication_status'] is
False:

    st.error('Username/password is incorrect')
```

```
elif st.session_state['authentication_status'] is None:
```

None:

```
    st.warning('Please enter your username and password')
```

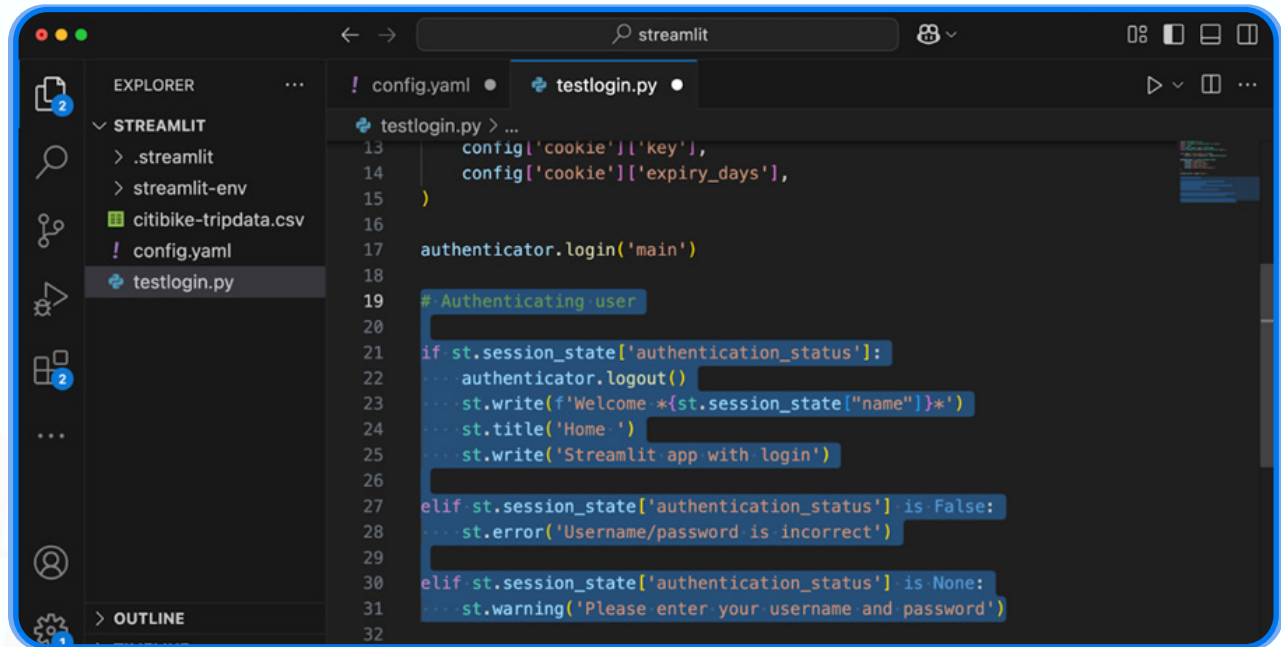


Figura 7. Edición del programa **testLogin.py**.

Paso 2. Prueba nuevamente tu aplicación con las nuevas funcionalidades.

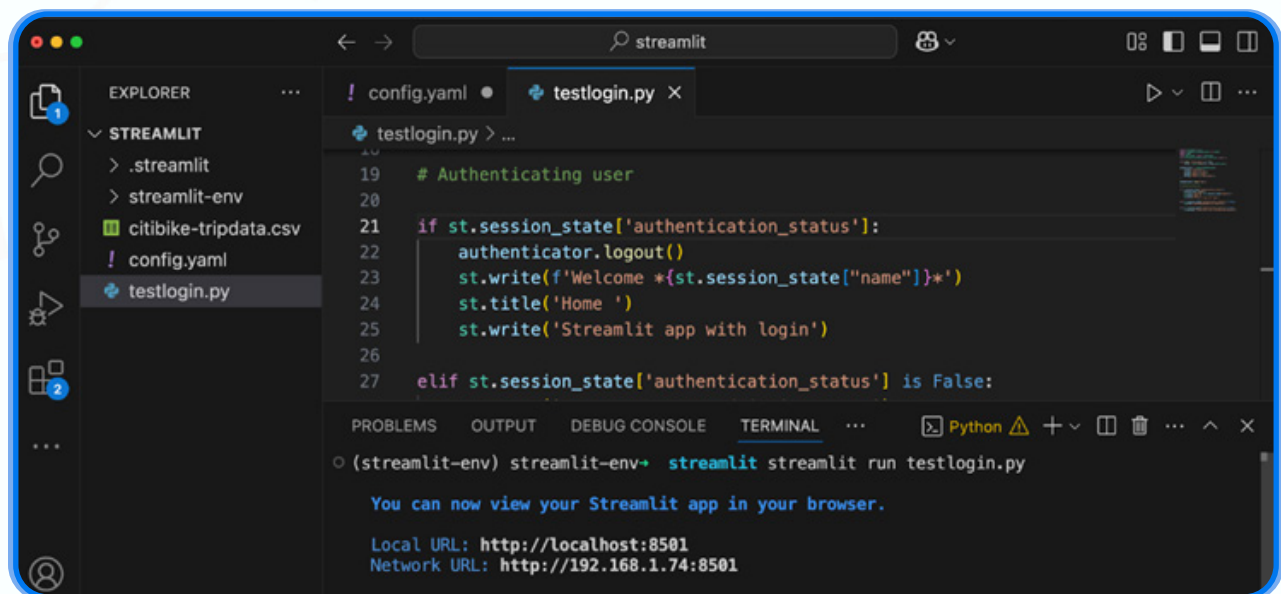


Figura 8. Prueba de aplicación.

Paso 3. Al hacerlo, abrirá el proyecto en **localhost:8501**.

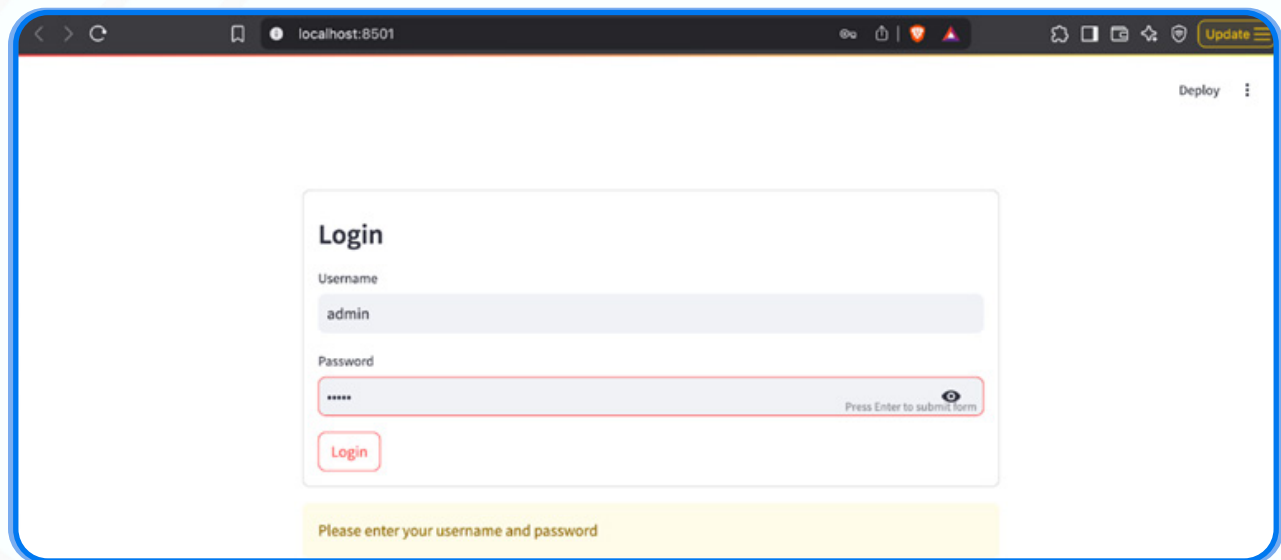


Figura 9. Vista del proyecto.

Paso 4. En este punto, ingresa **admin** como usuario y contraseña, después, da clic en el botón **Login**. Al hacerlo, aparecerá la pantalla de **Home** de tu aplicación con la bienvenida al usuario activo **admin**. Además, se mostrará el botón de **Logout**.

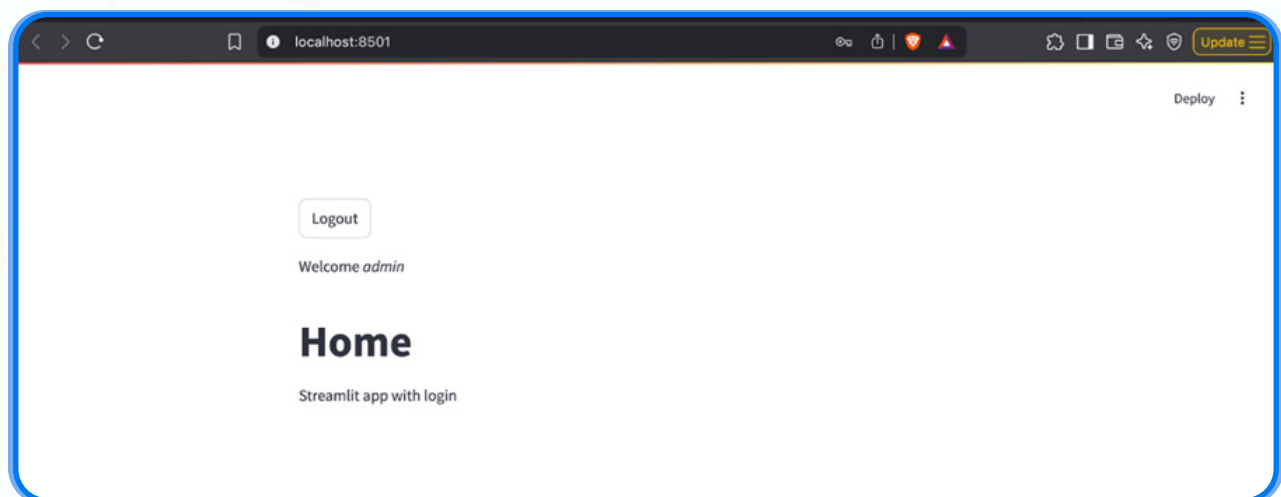


Figura 10. Vista del proyecto.

Paso 5. Prueba la aplicación haciendo clic en el botón **Logout**, al hacerlo, destruirá la sesión activa y te regresará a la pantalla de **Login**.

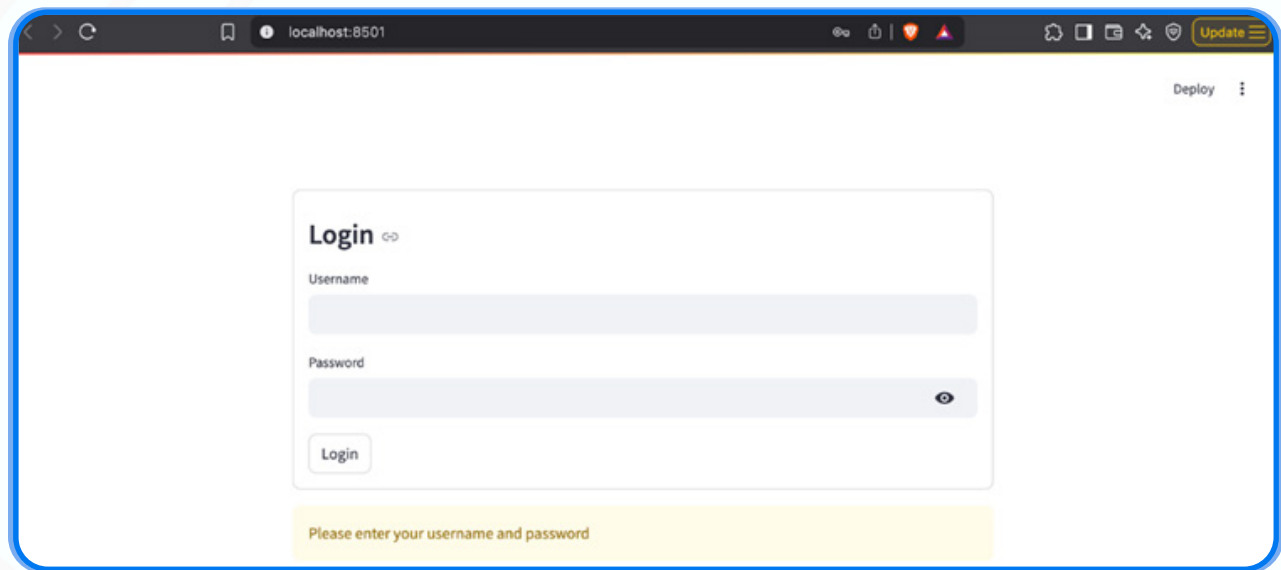


Figura 11. Vista de la sesión activa.

Hasta ahora has implementado un proyecto con autenticación básica por usuario y contraseña, **con operaciones de *Login*, *Logout*, validación de usuarios y redireccionamiento al *Home* de la aplicación.**

Pruébate

Ejercicio guiado

Desarrolla un código para una aplicación web que permita visualizar en un mapa de la ciudad de Nueva York los viajes realizados por la empresa Uber. Puedes descargar la base de datos **uber_dataset.csv** en el apartado de archivos adjuntos de la plataforma.

El conjunto de datos de Uber contiene los siguientes campos:

Date/Time - Fecha y hora de inicio de viaje

Lat - Latitud

Lon - Longitud

Base - Clave de base de operación

Instrucciones: Genera la estructura de la aplicación web para que muestre los siguientes elementos:

- Un título.
- Un subtítulo (este puede ser opcional).
- Un breve párrafo que describa el objetivo de este proyecto de analítica de datos; por ejemplo: Viajes de Uber en la ciudad de Nueva York con filtros por hora.
- Un control *slider* que permita filtrar los viajes por hora del día, usar la columna `Date/Time`.
- Una sección que contenga el mapa.

Considera crear una función para la obtención de datos, por ejemplo:

```
@st.cache

def load_data(nrows):

    data = pd.read_csv(DATA_URL, nrows=nrows)

    lowercase = lambda x: str(x).lower()

    data.rename(lowercase, axis='columns',
inplace=True)

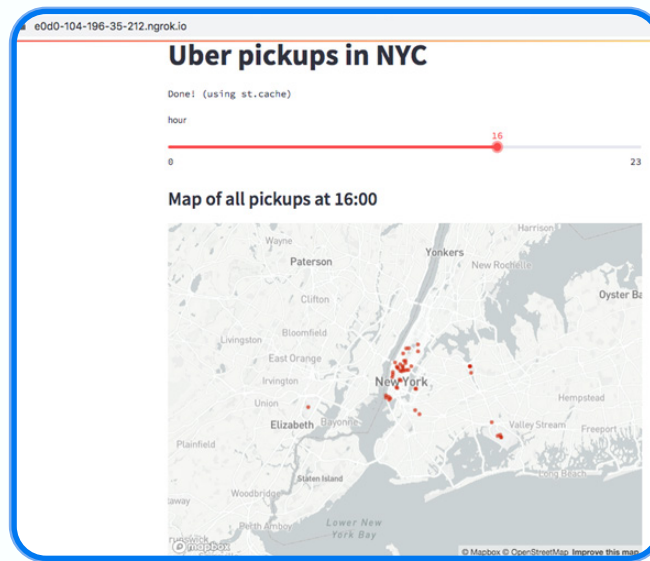
    data[DATE_COLUMN] = pd.to_datetime(data[DATE_
COLUMN])

    return data
```

La base de datos es muy grande, te sugerimos recuperar solo una muestra: `data = load_data(10000)` # probar con 100, 1000, etc

La columna `Date/Time` es de tipo Fecha y hora, te sugerimos aplicar un filtro por hora: `filtered_data = data[data[DATE_COLUMN].dt.hour == hour_to_filter]`

El resultado esperado es una aplicación de **Streamlit** que demuestre el uso de `st.map` con el *dataset* de Uber:



Cuestionario

Instrucciones: Lee con atención cada pregunta y selecciona una respuesta.

1. ¿Cuáles son las columnas que reconoce el componente de **Streamlit** `st.map` como puntos GPS?

a. `latitud`, `longitud`

b. `gps_lat`, `gps_lon`

c. `lat`, `lon`

Retroalimentación: `lat`, `lon` son las columnas que `st.map()` reconoce como puntos GPS.

2. ¿Cuál es el componente de **Python** que permite implementar operaciones de **login, logout**?

- a. `streamlit_comprobator`
- b. `streamlit_verificator`
- c. `streamlit_authenticator`

Retroalimentación: `streamlit_authenticator` es una de las bibliotecas en **Python** que permite implementar autenticación de usuarios.

3. ¿Qué archivo de configuración es adecuado para el módulo `streamlit_authenticator`?

- a. `auth.yaml`
- b. `config.yaml`
- c. `Dockerfile`

Retroalimentación: En la práctica, el archivo `config.yaml` es adecuado para guardar la configuración de usuarios y *passwords*.

4. ¿Cuál es la instrucción para instalar el módulo `streamlit_authenticator`?

- a. `pip install streamlitauthenticator`
- b. `pip install streamlit-authenticator`
- c. `pip install streamlit_authenticator`

Retroalimentación: La sintaxis `pip install streamlit-authenticator` permite instalar `streamlit_authenticator`.

Ideas para llevar

- Reconociste que en ciencia de datos se necesitan aplicaciones de *frontend* web y/o móvil para visualizar datos para aportar significado a la información.
- Identificaste que las aplicaciones web permiten generar nuevas oportunidades, solucionar problemas sustanciales y apoyar la toma de decisiones estratégicas.
- Revisaste como construir y visualizar aplicaciones web que acceden a *datasets* de información.
- Interiorizaste que como desarrollador de aplicaciones web, los *dashboards* son una herramienta valiosa en proyectos de ciencia de datos.

Bibliografía

Chen, A. (2021). *Use Streamlit and Firestore to create a serverless web app with persistent data, written entirely in Python* [Utiliza Streamlit y Firestore para crear una aplicación web sin servidor con datos persistentes, ¡escrita completamente en Python!]. Streamlit. Recuperado el 6 de junio de 2025 de: <https://blog.streamlit.io/streamlit-firestore/>

Ou, S. (2025). *Display Live Data Using Streamlit and a PostgreSQL Database* [Visualización de datos en directo mediante Streamlit y una base de datos PostgreSQL]. Ploomber. Recuperado el 6 de junio de 2025 de: <https://ploomber.io/blog/streamlit-postgres/>

Streamlit. (2025). *Connect Streamlit to MongoDB* [Conectar Streamlit a MongoDB]. Streamlit. Recuperado el 6 de junio de 2025 de: <https://docs.streamlit.io/develop/tutorials/databases/mongodb>

Streamlit. (2025). *Connect Streamlit to PostgreSQL* [Conectar Streamlit a PostgreSQL]. Streamlit. Recuperado el 6 de junio de 2025 de: <https://docs.streamlit.io/develop/tutorials/databases/postgresql>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 06 de junio de 2025 y su disponibilidad depende del autor.